



Clase 9

Infraestructura en Nube con IaC e IA Generativa

El punto de partida

El producto está construido, orquestado y con QA aprobado. La pregunta ahora es otra.

- ▶ **Clases 4-5:** specs + ADRs con AI-DLC · **Clase 6:** scaffolding y arneses
- ▶ **Clase 7:** orquestación AI-DLC → Linear, code review y memoria
- ▶ **Clase 8:** QA funcional cubierto (E2E · API · agentes)
- ▶ Tenemos código que funciona, está revisado, documentado y testeado
- ▶ **Lo que falta:** infraestructura reproducible — sin IaC, "funciona en mi máquina" es el estado del arte



El problema no es el código. Es que la infraestructura vive en la memoria del que hizo el deploy.

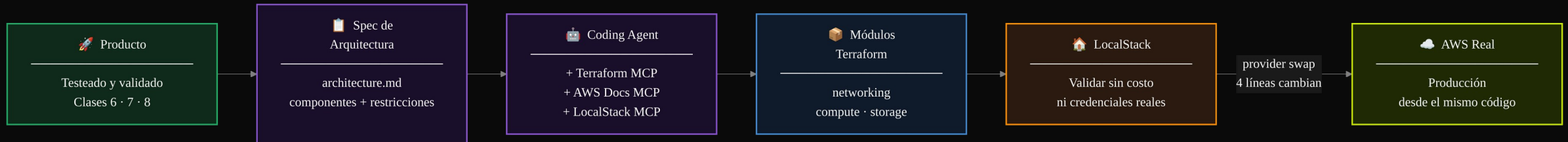


DEMO

Abrir localhost:3000 → producto en funcionamiento → `npm test` → tests pasan → preguntar: ¿cómo llega esto a producción hoy?

El camino de hoy

De producto local a producción en la nube



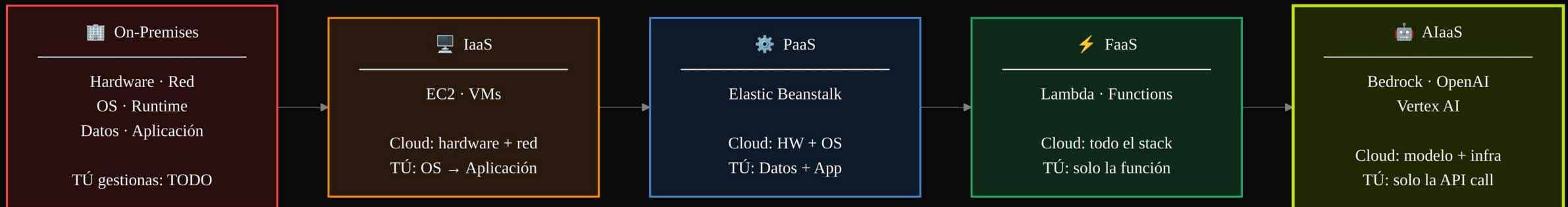
02 — SECCIÓN

Computación en la Nube

¿Qué alquila la nube y qué sigue siendo tuyo?

Modelos de servicio

A medida que subes en el stack, el proveedor asume más responsabilidades



💡 Elegir el nivel correcto no es una decisión técnica — es una decisión de cuánta responsabilidad operacional quieres asumir.

Proveedores principales

AWS, Azure, GCP — ¿cuándo elegir cada uno?

- ▶ **AWS:** líder en servicios gestionados y profundidad de features — default para la mayoría de cargas enterprise
- ▶ **Azure:** ventaja en entornos Microsoft/AD — fuerte en integración con on-premises
- ▶ **GCP:** ventaja en datos y ML — BigQuery, Vertex AI, GKE son best-in-class
- ▶ **Alternativas PaaS:** Railway, Fly.io, Vercel — cuando el time-to-market importa más que el control granular
- ▶ **Cuándo usar AIaaS:** Bedrock, Azure OpenAI, Vertex AI — IA gestionada sin operar GPUs

Democratización tecnológica

La nube eliminó las barreras de entrada a la infraestructura avanzada

- ▶ **Antes:** contact center → inversión de millones en hardware y licencias
- ▶ **Hoy:** Amazon Connect → una cuenta de AWS y configuración en horas
- ▶ **Antes:** data lake → equipo de 10 ingenieros de datos, 6 meses de proyecto
- ▶ **Hoy:** S3 + Glue + Athena → un arquitecto, 2 semanas
- ▶ **Antes:** modelo de IA → PhDs, GPUs propias, 2 años de investigación
- ▶ **Hoy:** Bedrock / Azure OpenAI → un developer, una llamada de API

💡 La nube no democratizó el acceso a servidores — democratizó el acceso a capacidades que antes eran exclusivas de grandes corporaciones.



DEMO

Consola AWS → EC2 (IaaS: ves el OS, instancias, networking) → Elastic Beanstalk (PaaS: solo subes el código) → Lambda (FaaS: solo escribes la función) → comparar densidad de configuración

03 — SECCIÓN

Infraestructura como Código

Infraestructura declarada en texto es infraestructura versionable, revisable y reproducible

¿Qué es IaC y por qué importa?

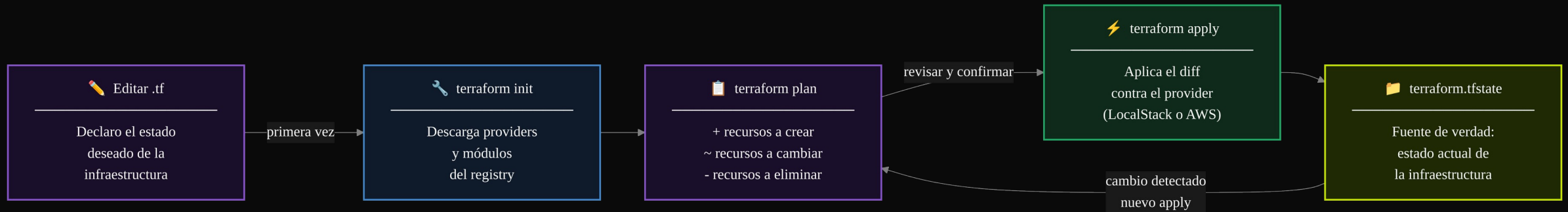
Infraestructura declarativa en lugar de instrucciones imperativas

- ▶ **Click-ops:** configurar la consola manualmente → no reproducible, no auditable, no versionable
- ▶ **Scripts imperativos:** secuencia de comandos AWS CLI → frágil cuando el estado ya existe
- ▶ **IaC declarativa:** describes el estado deseado → el tool calcula y aplica el diff
- ▶ **Idempotente:** ejecutar `apply` dos veces produce el mismo resultado
- ▶ **Auditable:** cada cambio de infraestructura tiene su commit, su PR, su revisión

💡 IaC no es automatizar clics — es describir lo que debería existir y dejar que la herramienta resuelva cómo llegar ahí.

El flujo de Terraform

init → plan → apply — predecible, explícito, reversible



El ecosistema de IaC

Terraform no está solo — cada herramienta tiene su nicho

- ▶ **Terraform (HCL):** multi-cloud, más de 3.000 providers, state management, plan/apply workflow
- ▶ **Pulumi:** misma potencia de Terraform pero con TypeScript, Python o Go en lugar de HCL
- ▶ **AWS CDK / CloudFormation:** nativo de AWS — más integración, menos portabilidad
- ▶ **Azure Bicep / ARM:** nativo de Azure — excelente para entornos Microsoft
- ▶ **Ansible / Chef:** configuración de instancias (qué instalar en el OS) — no confundir con IaC de infraestructura

💡 Elegimos Terraform por portabilidad: el mismo bloque de networking funciona en AWS, Azure y GCP con cambios mínimos.



DEMO

Abrir ``infra/main.tf`` → módulos (networking, storage, compute) → ``infra/modules/networking/main.tf`` → VPC con subnets → terminal: ``terraform plan -var-file=providers-local.tfvars`` → leer el diff

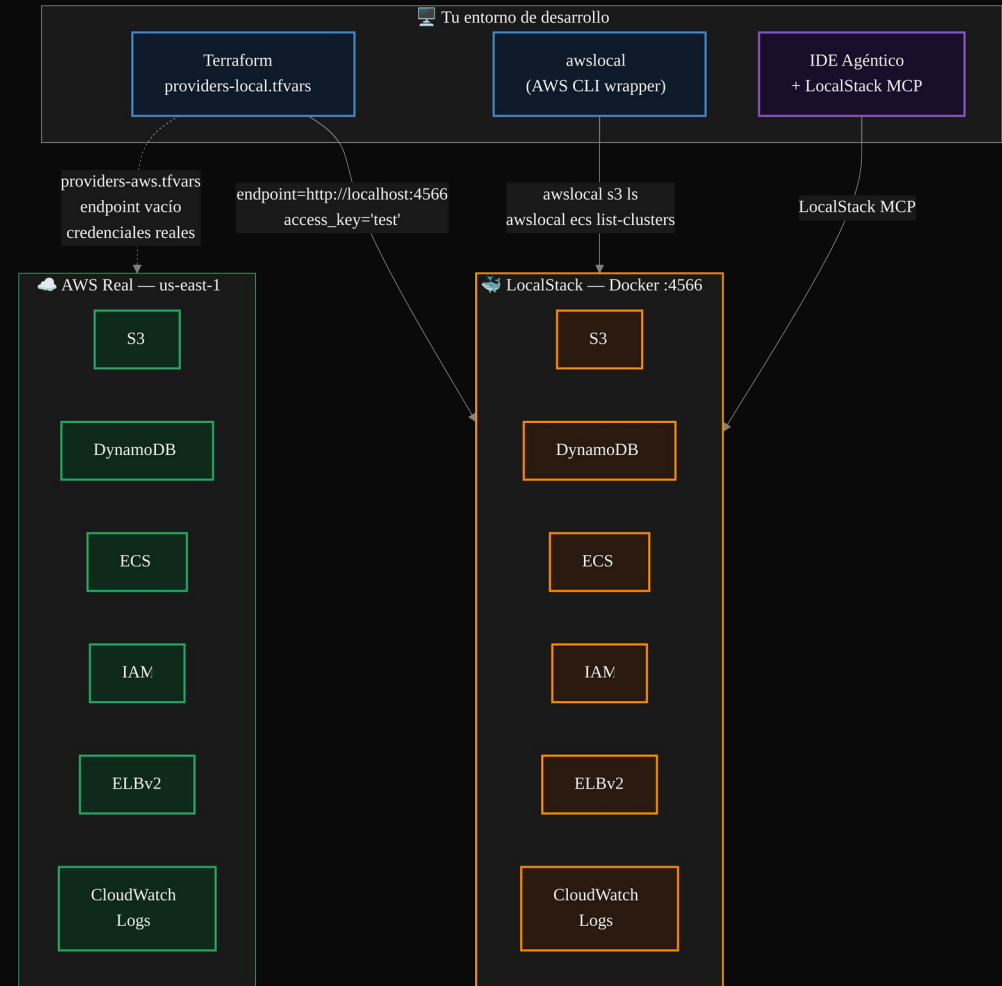
04 — SECCIÓN

LocalStack: La Nube en tu Máquina

Eliminar el costo y el riesgo del ciclo de desarrollo de infraestructura

Cómo funciona LocalStack

El mismo código apunta al emulador o a AWS con un solo cambio de configuración



El código de Terraform es 100% idéntico en LocalStack y en AWS real. Lo único que cambia son 4 líneas en el archivo de variables.

LocalStack en el flujo de trabajo

Local → validate → AWS: el ciclo que elimina sorpresas costosas

- ▶ **Desarrollo rápido:** crear y destruir recursos en segundos sin esperar a AWS
- ▶ **Sin costos:** S3, DynamoDB, ECS, Lambda, ALB — emulados sin facturación
- ▶ **Sin credenciales reales:** el equipo desarrolla con ``access_key = "test"``
- ▶ **CI/CD local:** pipelines de infraestructura se pueden testear en la laptop
- ▶ **El costo de un error en LocalStack:** 0€ · **el de un error en AWS:** tiempo + dinero + posible downtime

**DEMO**

``docker compose up -d` → `docker ps` (container `localstack-asistente-ia` healthy) → `localhost:4566/_localstack/health` → `awslocal s3 ls` (vacío) → `awslocal dynamodb list-tables` (sin tablas)`

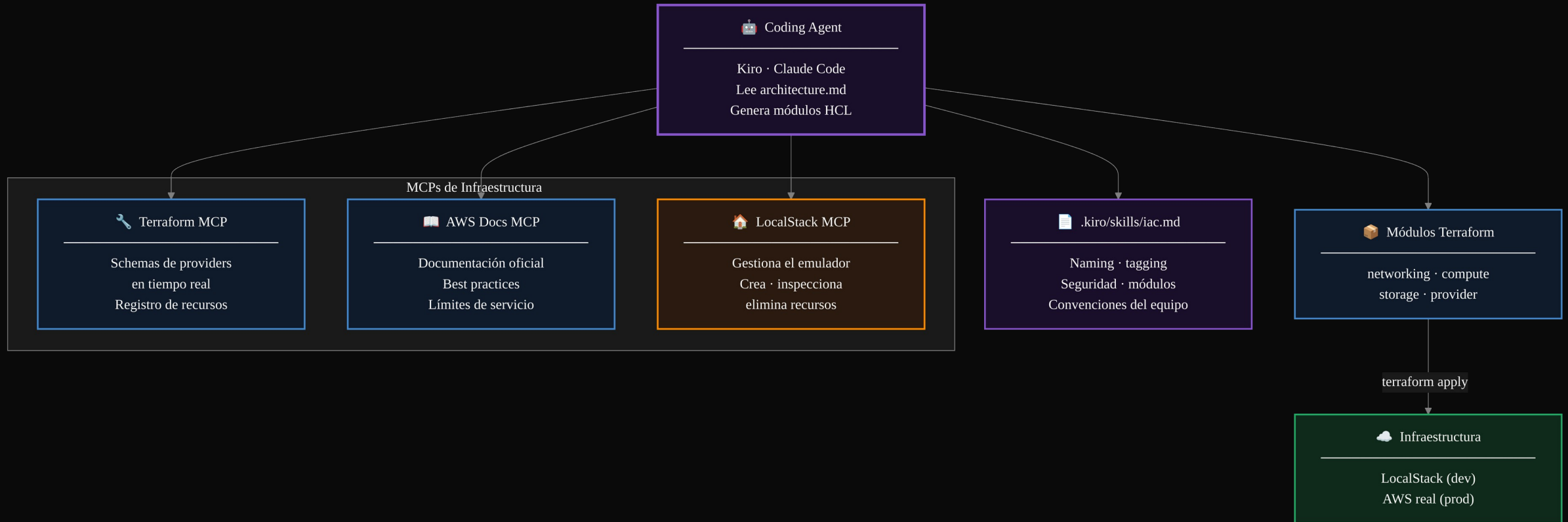
05 — SECCIÓN

IaC con IDEs Agénticos y MCPs

El agente como ingeniero de infraestructura que consulta el schema real antes de generar HCL

El stack de MCPs para infraestructura

Tres MCPs que convierten al coding agent en un experto de Terraform y AWS



Los tres MCPs de infraestructura

Cada MCP extiende las capacidades del agente en una dimensión distinta

- ▶ **Terraform MCP Server** (HashiCorp): schemas en tiempo real de providers y recursos — el agente genera HCL preciso, no código de memoria
- ▶ **AWS Documentation MCP** (awslabs): best practices, límites de servicio y cuotas — el agente valida antes de ``apply``
- ▶ **LocalStack MCP Server**: el agente crea, inspecciona y elimina recursos del emulador directamente desde el chat

💡 Sin MCPs, el agente genera HCL de memoria — puede estar desactualizado. Con Terraform MCP, el agente consulta el registry real en el momento de generar el código.



DEMO

Chat: "¿qué campos requiere ``aws_ecs_task_definition``?" → agente invoca Terraform MCP → schema completo → comparar con ``infra/modules/compute/main.tf``

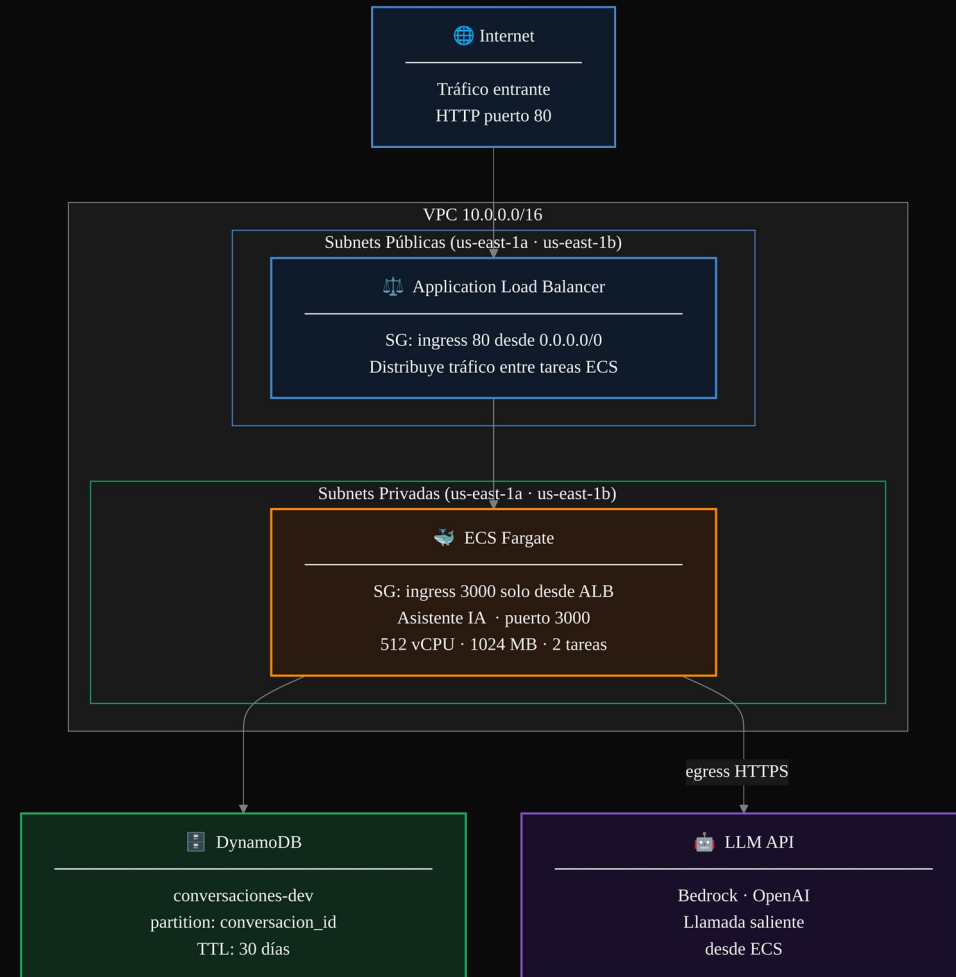
06 — SECCIÓN

Diseño de Infraestructura

La spec de arquitectura como contrato entre el equipo y el coding agent

Arquitectura del producto

*Lo que vamos a desplegar hoy:
networking + compute + storage*

**DEMO**

Abrir `specs/architecture.md` → componentes + relaciones + restricciones de seguridad → chat: "analiza esta spec y lista los módulos Terraform que necesitamos" → networking, compute, storage y dependencias

TRANSICIÓN

Kahoot

Validar comprensión antes del hands-on

- ▶ Modelos de servicio cloud: IaaS, PaaS, FaaS, AIaaS
- ▶ Flujo de Terraform: init → plan → apply
- ▶ LocalStack: qué emula y cuándo usarlo
- ▶ Terraform MCP · IaC declarativa vs imperativa

Para la próxima sesión

Infraestructura como código para tu producto en LocalStack

- ▶ ☐ **Spec de arquitectura** — ``architecture.md`` con componentes, relaciones y restricciones de seguridad
- ▶ ☐ **Módulos Terraform** — al menos ``networking``, ``compute`` y ``storage`` generados con el agente + Terraform MCP
- ▶ ☐ **LocalStack corriendo** — ``docker compose up -d`` con tu stack desplegado vía ``terraform apply`` contra ``localhost:4566``
- ▶ ☐ **Provider swap documentado** — ``providers-local.tfvars`` vs ``providers-aws.tfvars`` listos para el día que despliegues a AWS real



El objetivo es que tu producto pase de "funciona en mi máquina" a "tengo la infra declarada, validada en LocalStack y lista para AWS". El swap a producción debe ser cambiar 4 líneas, no rehacer la infra.